

Pipeline CI/CD - JobbingTrack

Vue d'ensemble

La pipeline CI/CD de JobbingTrack est une solution complète et automatisée qui garantit la qualité, la sécurité et la fiabilité du code à chaque modification. Elle intègre des analyses de sécurité, des tests complets, et des validations de qualité du code.

Approche recommandée : Utilisation des commandes Make

La pipeline CI/CD de JobbingTrack utilise un système de **commandes Make** centralisées qui orchestrent automatiquement tous les tests et validations. Cette approche est **recommandée** car elle :

-  **Automatise** l'environnement de test (Docker, DB, services)
-  **Centralise** la logique CI/CD
-  **Gère** les dépendances et l'ordre d'exécution
-  **Produit** des rapports cohérents
-  **Évite** les erreurs de configuration

Commandes Make principales pour CI/CD

```
# Tests complets (recommandé pour CI/CD)
make test-all

# Tests rapides (développement quotidien)
make test-quick

# Tests backend uniquement
make test-backend-only

# Tests frontend uniquement
make test-frontend-only

# Tests d'intégration système
make test-integration

# Tests de sécurité et performance
make test-security
make test-performance

# Tests avec coverage
make test-coverage

# Tests E2E complets
```

```
make test-e2e

# Tests d'accessibilité
make test-a11y

# Diagnostic et debugging
make diagnostic
make logs
make health
```

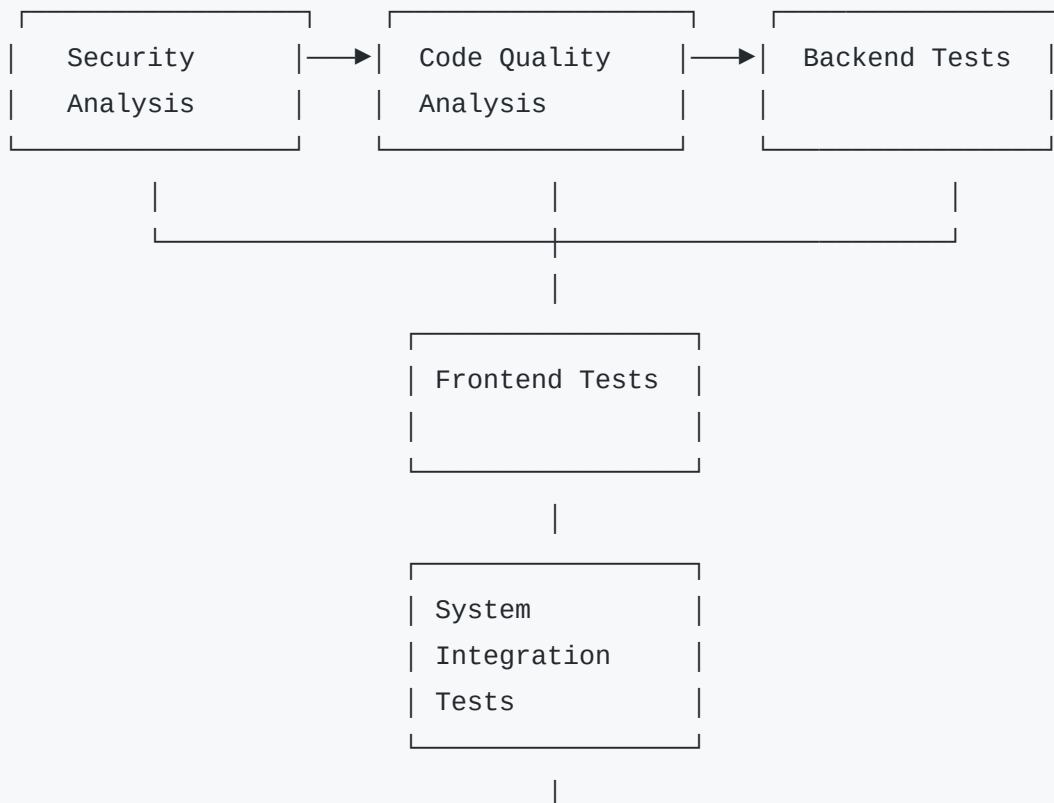
Alternative : Commandes directes

Si les commandes Make ne sont pas disponibles dans l'environnement CI/CD :

```
# Commandes npm équivalentes
npm run test:*
node tests/run-tests.js
./scripts/testing/run-tests.sh
```

Note : La pipeline GitHub Actions utilise les commandes Make quand disponibles, avec des fallbacks npm si nécessaire.

Architecture de la pipeline



Jobs de la pipeline

1. Security Analysis (security-scan)

Objectif : Identifier les vulnérabilités de sécurité avant l'exécution des tests.

Actions :

- Audit des dépendances npm avec `npm audit`
- Scan des vulnérabilités frontend et backend
- Détection des packages obsolètes ou malveillants

Configuration :

```
runs-on: ubuntu-latest
timeout-minutes: 15
```

Artefacts :

- Rapport de sécurité JSON
- Logs d'audit détaillés

2. Code Quality Analysis (code-quality)

Objectif : Garantir la qualité et la cohérence du code.

Actions :

- **ESLint** : Analyse statique du code JavaScript/TypeScript
- **Prettier** : Vérification du formatage du code
- **TypeScript** : Vérification des types et compilation
- **Backend** : Linting des services Node.js

Configuration :

```
runs-on: ubuntu-latest
timeout-minutes: 15
needs: security-scan
```

Règles vérifiées :

- Conventions de nommage
- Complexité cyclomatique
- Code non utilisé
- Bonnes pratiques de sécurité
- Formatage cohérent

3. Backend Tests (test-backend)

Objectif : Valider le bon fonctionnement des services backend.

Actions :

- **Tests unitaires** : Jest avec Supertest
- **Tests d'intégration** : Tests API complets
- **Tests de base de données** : PostgreSQL et Redis
- **Tests de santé** : Vérification des microservices

Configuration :

```
runs-on: ubuntu-latest
timeout-minutes: 30
needs: code-quality

services:
  postgres:
    image: postgres:15-alpine
  redis:
    image: redis:7-alpine
```

Couverture de test requise :

- Branches: $\geq 70\%$
- Functions: $\geq 70\%$
- Lines: $\geq 70\%$
- Statements: $\geq 70\%$

4. Frontend Tests (test-frontend)

Objectif : Valider l'interface utilisateur et l'expérience utilisateur.

Actions :

- **Tests unitaires** : Jest avec React Testing Library
- **Tests E2E** : Playwright multi-navigateurs
- **Tests mobile** : Simulation d'appareils mobiles
- **Tests d'accessibilité** : axe-core et WCAG 2.1

- **Build de production** : Next.js optimization

Configuration :

```
runs-on: ubuntu-latest
timeout-minutes: 25
needs: code-quality
```

Navigateurs testés :

- Chromium (Desktop & Mobile)
- Firefox (Desktop & Mobile)
- WebKit/Safari (Desktop & Mobile)

5. System Integration Tests (`system-integration-tests`)

Objectif : Valider l'intégration complète du système.

Actions :

- **Tests de santé système** : Vérification des services Docker
- **Tests d'intégration** : Workflows complets
- **Tests de performance** : Charge et temps de réponse

Configuration :

```
runs-on: ubuntu-latest
timeout-minutes: 20
needs: [test-backend, test-frontend]
```

6. CI/CD Summary (`ci-summary`)

Objectif : Générer un rapport final et des artefacts.

Actions :

- **Collecte des rapports** : Récupération de tous les résultats
- **Analyse des échecs** : Identification des causes
- **Génération d'artefacts** : Rapports JSON et HTML
- **Notifications** : Résumé des résultats

Configuration :

```
runs-on: ubuntu-latest
needs: [security-scan, code-quality, test-backend, test-frontend, system-integration-tests]
```

```
if: always()
```

Configuration des outils

ESLint

Backend (.eslinttrc.js) :

```
module.exports = {  
  env: {  
    node: true,  
    es2022: true,  
    jest: true,  
  },  
  extends: [  
    'eslint:recommended',  
    '@typescript-eslint/recommended',  
  ],  
  rules: {  
    'no-console': 'warn',  
    'no-unused-vars': ['warn', { argsIgnorePattern: '^_' }],  
    'prefer-const': 'error',  
    // ... autres règles  
  }  
};
```

Frontend (.eslinttrc.json) :

```
{  
  "extends": ["next/core-web-vitals"],  
  "rules": {  
    "@typescript-eslint/no-unused-vars": "warn",  
    "@typescript-eslint/no-explicit-any": "warn",  
    "react-hooks/exhaustive-deps": "warn"  
  }  
}
```

Prettier

Configuration (.prettierrc) :

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false
}
```

Jest

Backend (jest.config.js) :

```
module.exports = {
  testEnvironment: 'node',
  collectCoverageFrom: [
    'src/**/*.js',
    '!src/**/*.test.js',
  ],
  coverageThreshold: {
    global: {
      branches: 70,
      functions: 70,
      lines: 70,
      statements: 70,
    },
  },
  setupFilesAfterEnv: ['<rootDir>/tests/setup.js'],
};
```

Frontend (jest.config.js) :

```
const nextJest = require('next/jest')

const createJestConfig = nextJest({
  dir: './',
})

const customJestConfig = {
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
  testEnvironment: 'jsdom',
  collectCoverageFrom: [
    'src/**/*.{js,jsx,ts,tsx}',
    '!src/**/*.d.ts',
  ],
}
```

```

],
coverageThreshold: {
  global: {
    branches: 70,
    functions: 70,
    lines: 70,
    statements: 70,
  },
},
},
}

module.exports = createJestConfig(customJestConfig)

```

Playwright

Configuration (playwright.config.ts) :

```

export default defineConfig({
  testDir: './tests/e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: [
    ['html'],
    ['json', { outputFile: 'test-results.json' }],
    ['junit', { outputFile: 'test-results.xml' }],
    process.env.CI ? ['github'] : ['list']
  ],
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    // Tests mobiles
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },
    {
      name: 'Mobile Safari',

```



```
use: { ...devices['iPhone 12'] },  
},  
],  
});
```

Scripts de test

Commandes principales (recommandées) :

```
# Tests complets via Makefile  
make test-all  
  
# Tests rapides (sans E2E)  
make test-quick  
  
# Tests backend uniquement  
make test-backend-only  
  
# Tests frontend uniquement  
make test-frontend-only  
  
# Tests d'intégration  
make test-integration  
  
# Tests E2E  
make test-e2e  
  
# Tests de sécurité et performance  
make test-security  
make test-performance  
  
# Alternative si make non disponible :  
npm run test:*
```

Backend

Commandes principales :

```
# Tests backend (recommandé)  
make test-backend-only  
  
# Tests unitaires  
make test-unit  
  
# Tests avec coverage
```

```
make test-coverage

# Linting et formatage
make lint
make format:check

# Alternative npm :
npm run test
npm run test:ci
npm run test:watch
npm run lint
npm run format
```

Frontend

Commandes principales :

```
# Tests frontend (recommandé)
make test-frontend-only

# Tests unitaires
make test-frontend

# Tests avec coverage
make test-coverage

# Tests E2E
make test-e2e

# Tests d'accessibilité
make test-a11y

# Build de production
make build

# Alternative npm :
npm run test
npm run test:ci
npm run test:e2e
npm run test:a11y
npm run build
```

Variables d'environnement

CI/CD

```
env:
  DOCKER_BUILDKIT: 1
  COMPOSE_DOCKER_CLI_BUILD: 1
  NODE_ENV: test
```

Services de test

```
services:
  postgres:
    image: postgres:15-alpine
    env:
      POSTGRES_DB: jobbingtrack_test
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123

  redis:
    image: redis:7-alpine
```

Artefacts générés

Rapports de test

```
ci-artifacts/
├─ summary.json          # Résumé JSON de la pipeline
├─ backend/
│   ├─ coverage/        # Rapports de coverage backend
│   └─ test-results.json # Résultats des tests backend
├─ frontend/
│   ├─ coverage/        # Rapports de coverage frontend
│   ├─ test-results.json # Résultats Playwright
│   └─ accessibility.json # Rapports d'accessibilité
└─ security/
    └─ audit.json        # Rapport d'audit de sécurité
```

Métriques collectées

- **Coverage** : Pourcentage de code testé
- **Performance** : Temps d'exécution des tests
- **Sécurité** : Nombre de vulnérabilités

- **Qualité** : Nombre d'erreurs ESLint/Prettier
- **Accessibilité** : Score WCAG 2.1

Déclencheurs

Push automatique

```
on:
  push:
    branches: [ main, develop, feat/*, fix/* ]
```

Pull Request

```
on:
  pull_request:
    branches: [ main, develop ]
```

Exécution manuelle

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment de déploiement'
        required: true
        default: 'staging'
```

Gestion des échecs

Types d'échecs

1. **Échecs de sécurité** : Vulnérabilités critiques détectées
2. **Échecs de qualité** : Erreurs ESLint/Prettier
3. **Échecs de tests** : Tests unitaires ou E2E en échec
4. **Échecs d'infrastructure** : Services non disponibles

Actions automatiques

- **Retry automatique** : 2 tentatives pour les jobs E2E
- **Notification** : Alertes Slack/Email sur échecs
- **Rollback** : Annulation du déploiement en cas d'échec
- **Rapport détaillé** : Logs complets et captures d'écran

Optimisations

Cache

```
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: ${ steps.node-version.outputs.node-version }
    cache: 'npm'
    cache-dependency-path: |
      frontend/package-lock.json
      backend/*/package-lock.json
      tests/package-lock.json
```

Exécution parallèle

- Tests backend et frontend en parallèle
- Tests multi-navigateurs parallèles
- Cache des dépendances npm
- Services Docker persistants

Ressources optimisées

```
runs-on: ubuntu-latest # Runners GitHub optimisés
timeout-minutes: 25     # Timeout adapté par job
```

Monitoring et alertes

Métriques collectées

- Temps d'exécution par job
- Taux de succès/échec
- Coverage de test

- Vulnérabilités de sécurité
- Performance des tests

Notifications

- **Slack** : Notifications en temps réel
- **Email** : Rapports quotidiens
- **GitHub** : Commentaires automatiques sur PR
- **Dashboard** : Métriques en temps réel

Bonnes pratiques

Développement

1. Tests avant commit

```
# Commandes Make (recommandées)
make test-quick
make lint
make format:check

# Alternative npm :
npm run test
npm run lint
npm run format:check
```

2. Coverage minimal

- Maintenir $\geq 70\%$ de coverage
- Tester les cas d'erreur
- Tests d'intégration pour les API

3. Qualité du code

- Respecter les règles ESLint
- Formatage Prettier
- Commentaires pour le code complexe

Revue de code

- **Tests** : Vérifier que les nouveaux tests passent
- **Coverage** : S'assurer que le coverage n'a pas baissé
- **Sécurité** : Vérifier l'absence de vulnérabilités
- **Performance** : Contrôler les impacts sur les performances

Support et troubleshooting

Problèmes courants

1. Tests qui timeout

- Augmenter `timeout-minutes`
- Vérifier les services de test
- Optimiser les requêtes lentes

2. Échecs de cache

- Nettoyer le cache npm
- Vérifier les `cache-dependency-path`
- Rebuild des services

3. Erreurs Docker

- Vérifier les ports disponibles
- Contrôler les volumes Docker
- Logs des conteneurs

Debug

```
# Logs détaillés
make logs

# Status des services
make status

# Health check
make health

# Diagnostic complet
make diagnostic
```

Évolution

Améliorations planifiées

- ☐ Intégration SonarQube
- ☐ Tests de charge automatisés
- ☐ Analyse de performance continue
- ☐ Tests de sécurité avancés (SAST/DAST)
- ☐ Support multi-environnements

Métriques cibles

- **Temps d'exécution** : < 15 minutes
- **Taux de succès** : > 95%
- **Coverage** : > 80%
- **Sécurité** : 0 vulnérabilité critique
- **Accessibilité** : Score WCAG AA

Guide de référence : Commandes CI/CD

Tests automatiques (CI/CD)

```
make test-all          # Pipeline complète recommandée
make test-coverage      # Tests avec rapports coverage
make test-quick         # Tests rapides (développement)
```

Tests spécialisés

```
make test-backend-only # Backend uniquement
make test-frontend-only # Frontend uniquement
make test-integration  # Tests d'intégration
make test-e2e          # Tests End-to-End
make test-a11y         # Tests d'accessibilité
```

Debugging et diagnostic

```
make diagnostic         # Diagnostic complet
make logs               # Logs temps réel
make status             # Status services
make health             # Health check
```

Alternatives npm

```
npm run test:*          # Tests spécifiques
node tests/run-tests.js # Script principal
./scripts/testing/run-tests.sh # Script shell
```


Recommandations

Pour le développement quotidien

```
make test-quick      # Rapide et efficace
```

Pour la validation avant commit

```
make test-all        # Tests complets  
make status           # Vérifier l'état
```

Pour l'intégration continue

```
make test-coverage    # CI/CD avec rapports
```

Pour le debugging

```
make diagnostic        # Problèmes complexes  
make logs              # Investigation
```

Ressources

- [Documentation GitHub Actions](#)
- [Guide ESLint](#)
- [Documentation Playwright](#)
- [Standards WCAG 2.1](#)
- [Jest Documentation](#)
- [Makefile Guide](#)
- [Testing Guide](#)
- [Development Workflow](#)

💡 **Astuce** : Utilisez `make help` pour voir toutes les commandes disponibles et `make help-test` pour l'aide spécifique aux tests.

Pour toute question ou amélioration, consultez l'équipe de développement ou créez une issue sur le repository.